IC Design Laboratory

EE - 652

Group no :- 5

Risc-V Five stage Pipelined architecture

Code Flow (click on the .sv file name and navigate to particular code)

```
top_tb (testbench.sv)
└── TEST : TOP_Group_no_5 (TOP_pipeline.sv)
    ├── Program_Counter : PC_Module (PC.sv)
    │   └── MUX_FETCH : Mux (Mux.sv)
    │
    ├── Fetch : Fetch_Cycle (Fetch_Cycle.sv)
    │   ├── PCAdder : PC_Adder (PC_Adder.sv)
    │   └── I_MEM : Instruction_Memory (Instruction_Memory.sv)
    │
    ├── Decode : Decode_Cycle (Decode_Cycle.v)
    │   ├── ctrl_unit : Control_Unit_Top (Control_Unit_Top.sv)
    │   │   ├── Main_Decoder : Main_Decoder (Main_Decoder.sv)
    │   │   └── ALU_Decoder : ALU_Decoder (ALU_Decoder.sv)
    │   ├── RegFile : Register_File (Register_File.sv)
    │   └── SignExtend : Sign_Extend (Sign_Extend.sv)
    │
    ├── Execute : Execute_Cycle (Execute_Cycle.sv)
    │   ├── ALU_E : ALU (ALU.sv)
    │   ├── ADDER_E : PC_Adder (PC_Adder.sv)
    │   ├── MUX3X1_3 : Mux_3_by_1 (Mux.sv)
    │   ├── MUX3X1_1 : Mux_3_by_1 (Mux.sv)
    │   └── MUX3X1_2 : Mux_3_by_1 (Mux.sv)
    │
    ├── Memory : Memory_Cycle (Memory_Cycle.sv)
    │   └── DataMem : Data_Memory (Data_Memory.sv)
    │
    ├── WriteBack : Writeback_Cycle (Writeback_Cycle.sv)
    │   └── result_mux : Mux_3_by_1 (Mux.sv)
    │
    └── HazardUnit : Hazard_Unit (Hazard_Unit.sv)
    ├── Instructions in HEX : (instrustions.data)
    ├── Reg File : (reg_file.data)
    └── Waveform (waveform)
```

System-Verilog Code

1) **Testbench**

```
module top_tb;
      reg rst;
      reg clk=0;
      integer i;

      TOP_Group_5 TEST(clk,rst);

      initial begin
          rst=0;
          #20;
          rst=1;
          #630 $finish;
      end

      always @(posedge clk) begin
          $display("[%0t] fetched instruction: %h",$time,
      TEST.Fetch.InstrD);
      end

      initial begin
          forever #5 clk=~clk;
      end
      initial begin
          $dumpfile("wave.vcd");
        $dumpvars(0);
      End

endmodule
```

2) **TOP_Pipeline**

```
module TOP_Group_no_5(clk,rst);
    input clk,rst;

    // Declaration of Interim Wires
    wire PCSrcE, RegWriteW, RegWriteE, ALUSrcAE, MemWriteE,
BranchE, RegWriteM, MemWriteM;
    wire [1:0] ResultSrcM, ResultSrcW, ResultSrcE, ALUSrcBE;
    wire JumpE;
    wire [3:0] ALUControlE;
    wire [4:0] RDE, RDM, RDW;
```

```
    wire [31:0] PCTargetE, PCPlus4F_Fed,InstrD, PCD, PCPlus4D,
ResultW, RD1E, RD2E, ImmExtE, PCE, PCPlus4E, PCPlus4M,
WriteDataM, ALUResultM;
    wire [31:0] PCPlus4W, ALUResultW, ReadDataW;
    wire [4:0] RS1_E, RS2_E;
    wire [1:0] ForwardBE, ForwardAE;

    wire [31:0] PC;

    wire [4:0] Rs1D,Rs2D;
    wire StallD, StallF, FlushD, FlushE;
    // Module Initiation
    // Fetch Stage

    PC_Module Program_Counter(
        .clk(clk),
        .rst(rst),
        .PC(PC),
        .PCSrcE(PCSrcE),
        .PCTargetE(PCTargetE),
        .PCPlus4F(PCPlus4F_Fed),
        .en(StallF)
    );

    Fetch_Cycle Fetch (
        .clk(clk),
        .rst(rst),
        .PCF(PC),
        .InstrD(InstrD),
        .PCD(PCD),
        .PCPlus4D(PCPlus4D),
        .PCPlus4F_Fed(PCPlus4F_Fed),
        .en(StallD),
        .clr(FlushD)
    );

    // Decode Stage
    Decode_Cycle Decode (
        .clk(clk),
        .rst(rst),
        .InstrD(InstrD),
        .PCD(PCD),
        .PCPlus4D(PCPlus4D),
        .RegWriteW(RegWriteW),
        .RDW(RDW),
```

```
    .ResultW(ResultW),
    .RegWriteE(RegWriteE),
    .ALUSrcAE(ALUSrcAE),
    .ALUSrcBE(ALUSrcBE),
    .JumpE(JumpE),
    .MemWriteE(MemWriteE),
    .ResultSrcE(ResultSrcE),
    .BranchE(BranchE),
    .ALUControlE(ALUControlE),
    .RD1E(RD1E),
    .RD2E(RD2E),
    .ImmExtE(ImmExtE),
    .RDE(RDE),
    .PCE(PCE),
    .PCPlus4E(PCPlus4E),
    .RS1E(RS1_E),
    .RS2E(RS2_E),
    .Rs1D(Rs1D),
    .Rs2D(Rs2D),
    .clr(FlushE)
);


// Execute Stage
Execute_Cycle Execute (
        .clk(clk),
        .rst(rst),
        .RegWriteE(RegWriteE),
        .ALUSrcAE(ALUSrcAE),
        .ALUSrcBE(ALUSrcBE),
        .JumpE(JumpE),
        .MemWriteE(MemWriteE),
        .ResultSrcE(ResultSrcE),
        .BranchE(BranchE),
        .ALUControlE(ALUControlE),
        .RD1E(RD1E),
        .RD2E(RD2E),
        .ImmExtE(ImmExtE),
        .RdE(RDE),
        .PCE(PCE),
        .PCPlus4E(PCPlus4E),
        .PCSrcE(PCSrcE),
        .PCTargetE(PCTargetE),
        .RegWriteM(RegWriteM),
        .MemWriteM(MemWriteM),
```

```
        .ResultSrcM(ResultSrcM),
        .RdM(RDM),
        .PCPlus4M(PCPlus4M),
        .WriteDataM(WriteDataM),
        .ALUResultM(ALUResultM),
        .ResultW(ResultW),
        .ForwardAE(ForwardAE),
        .ForwardBE(ForwardBE)
    );

// Memory Stage
Memory_Cycle Memory (
    .clk(clk),
    .rst(rst),
    .RegWriteM(RegWriteM),
    .MemWriteM(MemWriteM),
    .ResultSrcM(ResultSrcM),
    .RdM(RDM),
    .PCPlus4M(PCPlus4M),
    .WriteDataM(WriteDataM),
    .ALUResultM(ALUResultM),
    .RegWriteW(RegWriteW),
    .ResultSrcW(ResultSrcW),
    .RdW(RDW),
    .PCPlus4W(PCPlus4W),
    .ALUResultW(ALUResultW),
    .ReadDataW(ReadDataW)
);

// Write Back Stage
Writeback_Cycle WriteBack (
    .clk(clk),
    .rst(rst),
    .ResultSrcW(ResultSrcW),
    .PCPlus4W(PCPlus4W),
    .ALUResultW(ALUResultW),
    .ReadDataW(ReadDataW),
    .ResultW(ResultW)
);

Hazard_Unit HazardUnit(
    .RegWriteM(RegWriteM),
    .RegWriteW(RegWriteW),
    .ResultSrcE0(ResultSrcE[0]),
    .PCSrcE(PCSrcE),
```

```
        .RdM(RDM),
        .RdW(RDW),
        .RdE(RDE),
        .Rs1E(RS1_E),
        .Rs2E(RS2_E),
        .Rs1D(Rs1D),
        .Rs2D(Rs2D),
        .ForwardAE(ForwardAE),
        .ForwardBE(ForwardBE),
        .StallD(StallD),
        .StallF(StallF),
        .FlushD(FlushD),
        .FlushE(FlushE)
    );

endmodule
```

3) **Program Counter Module**

```
module PC_Module(clk,rst,PC, PCSrcE,PCTargetE, PCPlus4F, en);
    input clk,rst;
    input en;
    wire [31:0]PC_Next;
    output reg [31:0]PC;
    input PCSrcE;
    input [31:0] PCPlus4F,PCTargetE;

    always @(posedge clk or negedge rst)
    begin
        if(rst == 1'b0)
            PC <= 32'd0;
        else if(!en)
            PC <= PC_Next;
        else
            PC <= 32'd0;
    end

    Mux MUX_FETCH(
        .a(PCPlus4F),
        .b(PCTargetE),
        .s(PCSrcE),
        .c(PC_Next)
```

```
    );
endmodule
```

4) **Mux 2x1**

```
module Mux (a,b,s,c);
    input [31:0]a,b;
    input s;
    output [31:0]c;

    assign c = (~s) ? a : b ;
endmodule
```

5) **Fetch_Cycle**

```
module Fetch_Cycle(clk,rst,PCF,InstrD,PCD,PCPlus4D, PCPlus4F_Fed,
en,clr);

input clk,rst;
input [31:0] PCF;
output [31:0] PCPlus4F_Fed;

input en, clr;

output [31:0] PCPlus4D, PCD, InstrD;

wire [31:0] PC_F,PCF;
wire [31:0] InstrF,PCPlus4F;

//REGs
reg [31:0] InstrF_reg, PCF_reg, PCPlus4F_reg;
assign PCPlus4F_Fed=PCPlus4F;

PC_Adder PCAdder(
    .a(PCF),
    .b(32'd4),
    .c(PCPlus4F)
);

Instruction_Memory I_MEM(
    .rst(rst),
    .A(PCF),
    .RD(InstrF)
);

always @(posedge clk or negedge rst) begin
```

```
        if(rst | ~clr | en) begin
            InstrF_reg<=InstrF;
            PCF_reg<=PCF;
            PCPlus4F_reg<=PCPlus4F;
        end
        else begin
            InstrF_reg<=32'd0;
            PCF_reg<=32'd0;
            PCPlus4F_reg<=32'd0;
        end
    end

    assign InstrD=InstrF_reg;
    assign PCD=PCF_reg;
    assign PCPlus4D=PCPlus4F_reg;

    endmodule
```

6) **PC_Adder**

```
    module PC_Adder (a,b,c);

        input [31:0]a,b;
        output [31:0]c;

        assign c = a + b;

    endmodule
```

7) **Instruction_Memory**

```
    module Instruction_Memory(rst,A,RD);

      input rst;
      input [31:0]A;
      output [31:0]RD;

      reg [31:0] mem [0:63];

      assign RD = (rst == 1'b0) ? {32{1'b0}} : mem[A[31:2]];

      initial begin
        $readmemh("instructions.data",mem);
      end

    endmodule
```

8) **Decode_Cycle**

```
module Decode_Cycle(InstrD,PCD,PCPlus4D,clk,rst, RegWriteW,
ResultW, RDW, //i/p
RegWriteE, ResultSrcE, MemWriteE, BranchE, ALUSrcAE, ALUSrcBE,
JumpE, //o/p
RD1E, RD2E, PCE, Rs1D, clr, Rs2D, RDE, ImmExtE, PCPlus4E,
ALUControlE, RS1E, RS2E);

input clk,rst;
input [31:0] InstrD, PCD, PCPlus4D;
input [31:0] ResultW;
input [4:0] RDW;
input RegWriteW;
input clr;

output [31:0] RD1E, RD2E, PCE, ImmExtE, PCPlus4E;
output [4:0] RDE;
output RegWriteE, MemWriteE, BranchE, ALUSrcAE;
output [1:0] ALUSrcBE;
output JumpE;
output [1:0] ResultSrcE;
output [3:0] ALUControlE;
output [4:0] RS1E, RS2E;
output [4:0] Rs1D, Rs2D;


//internal wires
wire RegWriteD,ALUSrcAD,MemWriteD,BranchD, JumpD;
wire [1:0] ALUSrcBD,ResultSrcD;
wire [2:0]ImmSrcD;
wire [3:0]ALUControlD;
wire [31:0] RD1D,RD2D,ImmExtD;
wire [4:0] RdD;
wire [4:0] RS1D, RS2D;
assign Rs1D=RS1D;
assign Rs2D=RS2D;
assign RS1D=InstrD[19:15];
assign RS2D=InstrD[24:20];

assign RdD=InstrD[11:7];

Control_Unit_Top ctrl_unit(
    .Op(InstrD[6:0]),
    .funct7(InstrD[31:25]),
    .funct3(InstrD[14:12]),
    .RegWrite(RegWriteD),
    .ImmSrc(ImmSrcD),
    .ALUSrcA(ALUSrcAD),
    .ALUSrcB(ALUSrcBD),
    .MemWrite(MemWriteD),
```

```
    .ResultSrc(ResultSrcD),
    .Branch(BranchD),
    .ALUControl(ALUControlD),
    .Jump(JumpD)
);

Register_File RegFile(
    .clk(clk),
    .rst(rst),
    .WE3(RegWriteW),
    .WD3(ResultW),
    .A1(InstrD[19:15]),
    .A2(InstrD[24:20]),
    .A3(RDW),
    .RD1(RD1D),
    .RD2(RD2D)
);

Sign_Extend SignExtend(
    .In(InstrD[31:7]),
    .ImmSrc(ImmSrcD),
    .Imm_Ext(ImmExtD)
);

//Registers
reg [31:0] RD1D_reg, RD2D_reg, PCD_reg, ImmExtD_reg,
PCPlus4D_reg;
reg [4:0] RDD_reg;
reg RegWriteD_reg, MemWriteD_reg, BranchD_reg, ALUSrcAD_reg;
reg [1:0] ALUSrcBD_reg;
reg [1:0] ResultSrcD_reg;
reg [3:0] ALUControlD_reg;
reg [4:0] RS1D_reg, RS2D_reg;
reg JumpD_reg;


always @(posedge clk or negedge rst) begin
    if(rst | (~clr)) begin
        RD1D_reg<=RD1D;
        RD2D_reg<=RD2D;
        PCD_reg<=PCD;
        ImmExtD_reg<=ImmExtD;
        PCPlus4D_reg<=PCPlus4D;
        RDD_reg<=RdD;
        RegWriteD_reg<=RegWriteD;
        MemWriteD_reg<=MemWriteD;
        BranchD_reg<=BranchD;
        ALUSrcAD_reg<=ALUSrcAD;
        ALUSrcBD_reg<=ALUSrcBD;
        JumpD_reg<=JumpD;
        ResultSrcD_reg<=ResultSrcD;
        ALUControlD_reg<=ALUControlD;
```

```
            RS1D_reg<=RS1D;
            RS2D_reg<=RS2D;
        end
        else begin
            RD1D_reg<=32'd0;
            RD2D_reg<=32'd0;
            PCD_reg<=32'd0;
            ImmExtD_reg<=32'd0;
            PCPlus4D_reg<=32'd0;
            RDD_reg<=32'd0;
            RegWriteD_reg<=1'd0;
            MemWriteD_reg<=1'd0;
            BranchD_reg<=1'd0;
            ALUSrcAD_reg<=1'd0;
            ALUSrcBD_reg<=2'd0;
            JumpD_reg<=1'd0;
            ResultSrcD_reg<=2'd0;
            ALUControlD_reg<=4'd0;
            RS1D_reg<=5'd0;
            RS2D_reg<=5'd0;
        end
    end

    assign RD1E=RD1D_reg;
    assign RD2E=RD2D_reg;
    assign PCE=PCD_reg;
    assign ImmExtE=ImmExtD_reg;
    assign PCPlus4E=PCPlus4D_reg;
    assign RDE=RDD_reg;
    assign RegWriteE=RegWriteD_reg;
    assign MemWriteE=MemWriteD_reg;
    assign BranchE=BranchD_reg;
    assign ALUSrcAE=ALUSrcAD_reg;
    assign ALUSrcBE=ALUSrcBD_reg;
    assign JumpE=JumpD_reg;
    assign ResultSrcE=ResultSrcD_reg;
    assign ALUControlE=ALUControlD_reg;
    assign RS1E=RS1D_reg;
    assign RS2E=RS2D_reg;
    // RS2D_reg<=RS2D;

    endmodule
```

9) **Control_Unit_Top**

```
module
Control_Unit_Top(Op,RegWrite,ImmSrc,ALUSrcA,ALUSrcB,MemWrite,Resu
ltSrc,
Branch,funct3,funct7,ALUControl,Jump);

    input [6:0]Op,funct7;
```

```
    input [2:0]funct3;
    output RegWrite,ALUSrcA,MemWrite,Branch, Jump;
    output [2:0]ImmSrc;
    output [3:0]ALUControl;
    output [1:0] ResultSrc, ALUSrcB;


    wire [1:0]ALUOp;

    Main_Decoder Main_Decoder(
                .Op(Op),
                .RegWrite(RegWrite),
                .ImmSrc(ImmSrc),
                .MemWrite(MemWrite),
                .ResultSrc(ResultSrc),
                .Branch(Branch),
                .ALUSrcA(ALUSrcA),
                .ALUOp(ALUOp),
                .Jump(Jump),
                .ALUSrcB(ALUSrcB)
    );

    ALU_Decoder ALU_Decoder(
                            .ALUOp(ALUOp),
                            .funct3(funct3),
                            .funct7(funct7),
                            .op(Op),
                            .ALUControl(ALUControl)
    );

endmodule
```

10) **Main_Decoder**

```
module Main_Decoder(
    input [6:0] Op,
    output [1:0] ResultSrc,
    output MemWrite,
    output Branch, ALUSrcA,
    output [1:0] ALUSrcB,
    output RegWrite, Jump,
    output [2:0] ImmSrc,
    output [1:0] ALUOp
);

reg [13:0] controls;
```

```
assign {RegWrite, ImmSrc, ALUSrcA, ALUSrcB, MemWrite, ResultSrc,
Branch, ALUOp, Jump} = controls;

always @ (*)
    case(Op)
    //
RegWrite_ImmSrc_ALUSrcA_ALUSrcB_MemWrite_ResultSrc_Branch_ALUOp_J
ump
        7'b0000011: controls = 14'b1_000_0_01_0_01_0_00_0; // lw
        7'b0100011: controls = 14'b0_001_0_01_1_00_0_00_0; // sw
        7'b0110011: controls = 14'b1_xxx_0_00_0_00_0_10_0; //
R-type
        7'b1100011: controls = 14'b0_010_0_00_0_00_1_01_0; //
B-type
        7'b0010011: controls = 14'b1_000_0_01_0_00_0_10_0; //
I-type ALU
        7'b1101111: controls = 14'b1_011_0_00_0_10_0_00_1; // jal
        7'b0010111: controls = 14'b1_100_1_10_0_00_0_00_0; //
auipc // PC Target for SrcB
        7'b0110111: controls = 14'b1_100_1_01_0_00_0_00_0; // lui
        7'b1100111: controls = 14'b1_000_0_01_0_10_0_00_1; //
jalr
        7'b0000000: controls = 14'b0_000_0_00_0_00_0_00_0; // for
default values on reset

        default:    controls = 14'bx_xxx_x_xx_x_xx_x_xx_x; //
instruction not implemented
    endcase
endmodule
```

11) **ALU_Decoder**

```
module ALU_Decoder(ALUOp,funct3,funct7,op,ALUControl);

    input [1:0]ALUOp;
    input [2:0]funct3;
    input [6:0]funct7,op;
    output reg [3:0]ALUControl;
//
wire RtypeSub;
assign RtypeSub = funct7[5] & op[5]; // TRUE for R-type subtract
always @ (*)
    case(ALUOp)
        2'b00: ALUControl = 4'b0000; // addition
        2'b01: ALUControl = 4'b0001; // subtraction
```

```
        default: case(funct3) // R-type or I-type ALU
                3'b000: if (RtypeSub)
                            ALUControl = 4'b0001; // sub
                        else
                            ALUControl = 4'b0000; // add, addi
                3'b001: ALUControl = 4'b0100; // sll, slli
                3'b010: ALUControl = 4'b0101; // slt, slti
                3'b011: ALUControl = 4'b1000; // sltu, sltiu
                3'b100: ALUControl = 4'b0110; // xor, xori
                3'b101: if (~funct7)
                            ALUControl = 4'b0111;   // srl
                        else
                            ALUControl = 4'b1111;  // sra
                3'b110: ALUControl = 4'b0011; // or, ori
                3'b111: ALUControl = 4'b0010; // and, andi
                default: ALUControl = 4'bxxxx;
                endcase
    endcase

endmodule
```

12) **Register_File**

```
module Register_File(clk,rst,WE3,WD3,A1,A2,A3,RD1,RD2);

    input clk,rst,WE3;
    input [4:0]A1,A2,A3;
    input [31:0]WD3;
    output [31:0]RD1,RD2;

    reg [31:0] Register [0:31];

    always @ (posedge clk)
    begin
        if(WE3 & (A3 != 5'h00))
            Register[A3] <= WD3;
    end

    assign RD1 = (rst==1'b0) ? 32'd0 : Register[A1];
    assign RD2 = (rst==1'b0) ? 32'd0 : Register[A2];

    initial begin
        Register[0] = 32'h00000000;
        for (integer i=1; i<32; i=i+1) begin
            Register[i]=32'hffff_ffff;
```

```
        end
    end

    always @(posedge clk) begin
        $writememh("reg.data",Register);
    end
endmodule
```

13) **Sign_Extend**

```
module Sign_Extend (input  [31:7] In,
            input  [2:0] ImmSrc,
            output reg [31:0] Imm_Ext);


    always @ (*)
        case(ImmSrc)
        // I-type
        3'b000: Imm_Ext = {{20{In[31]}}, In[31:20]};

        // S-type (stores)
        3'b001: Imm_Ext = {{20{In[31]}}, In[31:25], In[11:7]};

        // B-type (branches)
        3'b010: Imm_Ext = {{20{In[31]}}, In[7], In[30:25],
In[11:8], 1'b0};

        // J-type (jal)
        3'b011: Imm_Ext = {{12{In[31]}}, In[19:12], In[20],
In[30:21], 1'b0};

        // U-type
        3'b100: Imm_Ext = {In[31:12], 12'b0};

        default: Imm_Ext = 32'bx; // undefined
    endcase
endmodule
```

14) **Execute_Cycle**

```
module Execute_Cycle(clk, rst, RegWriteE, ResultSrcE, MemWriteE,
BranchE, ALUControlE, ALUSrcAE, ALUSrcBE, JumpE,
RD1E,RD2E, PCE, RdE, ImmExtE,PCPlus4E, PCSrcE, PCTargetE,
RegWriteM, ResultSrcM, MemWriteM, ALUResultM,
WriteDataM, RdM, PCPlus4M, ResultW, ForwardAE, ForwardBE);
```

```
//IO
input clk, rst, RegWriteE, MemWriteE, BranchE;
input [3:0] ALUControlE;
input ALUSrcAE, JumpE;
input [1:0] ALUSrcBE,ResultSrcE;
input [4:0] RdE;
input [31:0] RD1E,RD2E, PCE, ImmExtE,PCPlus4E, ResultW;
input [1:0] ForwardAE, ForwardBE;

output PCSrcE, RegWriteM, MemWriteM;
output [31:0] ALUResultM, WriteDataM, PCPlus4M, PCTargetE;
output [4:0] RdM;
output [1:0] ResultSrcM;

//Internal wires
wire [31:0] SrcAE;
wire [31:0] SrcBE;
wire [31:0] ALUResultE;
wire ZeroE;
wire [31:0]RD2E_Mux;
wire [31:0]SrcAE_Inter;

//Regs
reg RegWriteM_reg, MemWriteM_reg;
reg [31:0] ALUResultM_reg, WriteDataM_reg, PCPlus4M_reg;
reg [4:0] RdM_reg;
reg [1:0] ResultSrcM_reg;

ALU ALU_E (
    .A(SrcAE),
    .B(SrcBE),
    .Result(ALUResultE),
    .ALUControl(ALUControlE),
    .OverFlow(),
    .Zero(ZeroE),
    .Negative()
);

PC_Adder ADDER_E(
    .a(PCE),
    .b(ImmExtE),
    .c(PCTargetE)
);

Mux_3_by_1 MUX3X1_3(
```

```
    .a(RD2E_Mux),
    .b(ImmExtE),
    .s(ALUSrcBE),
    .c(PCTargetE),
    .d(SrcBE)
);


Mux_3_by_1 MUX3X1_1(
    .a(RD1E),
    .b(ResultW),
    .c(ALUResultM),
    .s(ForwardAE),
    .d(SrcAE_Inter)
);

assign SrcAE=(ALUSrcAE) ? 32'b0:SrcAE_Inter;

Mux_3_by_1 MUX3X1_2(
    .a(RD2E),
    .b(ResultW),
    .c(ALUResultM),
    .s(ForwardBE),
    .d(RD2E_Mux)
);




always @(posedge clk or negedge rst) begin
    if (rst) begin
        RegWriteM_reg<=RegWriteE;
        ResultSrcM_reg<=ResultSrcE;
        MemWriteM_reg<=MemWriteE;
        ALUResultM_reg<=ALUResultE;
        WriteDataM_reg<=RD2E_Mux;
        RdM_reg<=RdE;
        PCPlus4M_reg<=PCPlus4E;
    end
    else begin
        RegWriteM_reg<=1'b0;
        ResultSrcM_reg<=2'd0;
        MemWriteM_reg<=1'd0;
        ALUResultM_reg<=32'd0;
        WriteDataM_reg<=32'd0;
```

```
        RdM_reg<=5'd0;
        PCPlus4M_reg<=32'd0;
    end
end

//Output Assignment
// assign PCSrcE=ZeroE&BranchE;
assign RegWriteM=RegWriteM_reg;
assign ResultSrcM=ResultSrcM_reg;
assign MemWriteM=MemWriteM_reg;
assign ALUResultM=ALUResultM_reg;
assign WriteDataM=WriteDataM_reg;
assign RdM=RdM_reg;
assign PCPlus4M=PCPlus4M_reg;

assign PCSrcE = (JumpE | (BranchE & ZeroE));

endmodule
```

15) **ALU**

```
module ALU(A,B,Result,ALUControl,OverFlow,Zero,Negative);

input  [31:0] A;
input  [31:0] B;
input [3:0] ALUControl;
output reg  [31:0] Result;
output wire Zero, Negative;
output reg OverFlow;

wire [31:0] Sum;


assign Sum = A + (ALUControl[0] ? ~B : B) + ALUControl[0];  //
sub using 1's complement


assign Zero = (|Result) ? 1'b0:1'b1;
assign Negative = Result[31];


always @ (*) begin
        OverFlow = ~(ALUControl[0] ^ B[31] ^ A[31]) & (A[31] ^
Sum[31]) & (~ALUControl[1]);
        casex (ALUControl)
```

```
                4'b0000: Result = Sum;              // sum or
diff
                4'b0001: Result = Sum;              // sum or
diff
                4'b0010: Result = A & B;    // and
                4'b0011: Result = A | B;    // or
                4'b0100: Result = A << B[4:0];  // sll, slli
                4'b0101: Result = {{30{1'b0}}, OverFlow ^
Sum[31]}; //slt, slti
                4'b0110: Result = A ^ B;   // Xor
                4'b0111: Result = A >> B[4:0];  // shift logic
                4'b1000: Result = ($unsigned(A) < $unsigned(B));
//sltu, stlui
                4'b1111: Result = A >>> B[4:0]; //shift
arithmetic
                default: Result = 32'd0;
        endcase

end

endmodule
```

16) **Mux_3_by_1**

```
module Mux_3_by_1 (a,b,c,s,d);
    input [31:0] a,b,c;
    input [1:0] s;
    output [31:0] d;

    assign d = (s == 2'b00) ? a : (s == 2'b01) ? b : (s == 2'b10)
? c : 32'h00000000;

endmodule
```

17) **Memory_Cycle**

```
module Memory_Cycle(
    clk, rst, RegWriteM, ResultSrcM, MemWriteM,
    ALUResultM, WriteDataM, RdM, PCPlus4M, ALUResultW,
    RegWriteW, ResultSrcW, ReadDataW, RdW, PCPlus4W
);

input clk,rst,RegWriteM,MemWriteM;
input [31:0] ALUResultM, WriteDataM, PCPlus4M;
input [4:0] RdM;
input [1:0] ResultSrcM;
```

```
output [31:0] ReadDataW, PCPlus4W, ALUResultW;
output RegWriteW;
output [1:0] ResultSrcW;
output [4:0]RdW;

wire [31:0] ReadDataM;

Data_Memory DataMem(
    .clk(clk),
    .rst(rst),
    .WE(MemWriteM),
    .WD(WriteDataM),
    .A(ALUResultM),
    .RD(ReadDataM)
);

reg [31:0] ReadDataW_reg, RdW_reg, PCPlus4W_reg, ALUResultW_reg;
reg RegWriteW_reg;
reg [1:0] ResultSrcW_reg;

always @(posedge clk or negedge rst) begin
    if(rst) begin
        ReadDataW_reg<=ReadDataM;
        RdW_reg<=RdM;
        PCPlus4W_reg<=PCPlus4M;
        ALUResultW_reg<=ALUResultM;
        RegWriteW_reg<=RegWriteM;
        ResultSrcW_reg<=ResultSrcM;
    end
    else begin
        ReadDataW_reg<=32'd0;
        RdW_reg<=5'd0;
        PCPlus4W_reg<=32'd0;
        ALUResultW_reg<=32'd0;
        RegWriteW_reg<=1'd0;
        ResultSrcW_reg<=2'd0;
    end
end

assign ReadDataW=ReadDataW_reg;
assign RdW=RdW_reg;
assign PCPlus4W=PCPlus4W_reg;
assign ALUResultW=ALUResultW_reg;
assign RegWriteW=RegWriteW_reg;
assign ResultSrcW=ResultSrcW_reg;
```

```
endmodule
```

18) **Data_Memory**

```
module Data_Memory(clk,rst,WE,WD,A,RD);

    input clk,rst,WE;
    input [31:0]A,WD;
    output [31:0]RD;

    reg [31:0] mem [0:1023];

    always @ (posedge clk)
    begin
        if(WE)
            mem[A] <= WD;
    end

    assign RD = (~rst) ? 32'd0 : mem[A];


    always @ (posedge clk) begin
        $writememh("dmemory.data",mem);
    end


endmodule
```

19) **Writeback_Cycle**

```
module Writeback_Cycle(
    clk, rst, ReadDataW, ResultSrcW, ALUResultW,
    PCPlus4W, ResultW
);

input clk, rst;
input [1:0] ResultSrcW;
input [31:0] PCPlus4W, ALUResultW, ReadDataW;

output [31:0] ResultW;

Mux_3_by_1 result_mux(
.a(ALUResultW),
.b(ReadDataW),
.c(PCPlus4W),
.s(ResultSrcW),
```

```
.d(ResultW)
);

endmodule
```

20) **Hazard_Unit**

```
module Hazard_Unit(input  [4:0] Rs1D, Rs2D, Rs1E, Rs2E,
                input  [4:0] RdE, RdM, RdW,
                input  RegWriteM, RegWriteW,
                    input  ResultSrcE0, PCSrcE, rst,
                output reg [1:0] ForwardAE, ForwardBE,
                output StallD, StallF, FlushD, FlushE);



wire lwStall;
always @(*) begin
ForwardAE = 2'b00;
    ForwardBE = 2'b00;
if ((Rs1E == RdM) & (RegWriteM) & (Rs1E != 0)) // higher priority
- most recent
    ForwardAE = 2'b10; // for forwarding ALU Result in Memory
Stage
else if ((Rs1E == RdW) & (RegWriteW) & (Rs1E != 0))
    ForwardAE = 2'b01; // for forwarding WriteBack Stage Result



if ((Rs2E == RdM) & (RegWriteM) & (Rs2E != 0))
    ForwardBE = 2'b10; // for forwarding ALU Result in Memory
Stage

else if ((Rs2E == RdW) & (RegWriteW) & (Rs2E != 0))
    ForwardBE = 2'b01; // for forwarding WriteBack Stage Result

end


   assign lwStall = (ResultSrcE0 == 1) & ((RdE == Rs1D) | (RdE ==
Rs2D));
//   assign FlushE = lwStall;
    assign StallF = lwStall & (rst);
    assign StallD = lwStall & rst;
```

```
    assign FlushE = (lwStall | PCSrcE) & rst;
    assign FlushD = PCSrcE & rst;

endmodule
```

21) **Instructions in HEX**

Hex code

```
00400293
00700313
00a00393
00300413
006284b3
007484b3
008484b3
```

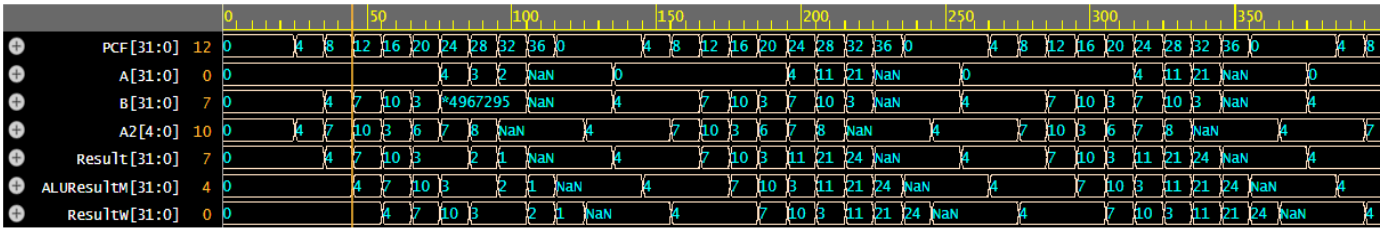
Assembly Code:-

```
addi x5, x0, 4      # x5 = 4
addi x6, x0, 7      # x6 = 7
addi x7, x0, 10     # x7 = 10
addi x8, x0, 3      # x8 = 3

add  x9, x5, x6    # x9 = x5 + x6 = 4 + 7 = 11
add  x9, x9, x7    # x9 = 11 + 10 = 21
add  x9, x9, x8    # x9 = 21 + 3 = 24
```

22) **Reg File**

```
00000000
ffffffff
ffffffff
ffffffff
ffffffff
00000002
00000005
00000008
fffffffe
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
ffffffff
```

```
ffffffff
ffffffff
ffffffff
0000000a
ffffffff
ffffffff
ffffffff
```

23) **Waveform**